DRIVER PORTAL

Business Requirements Document

Self-Initiated Portfolio Project | Version 2.2

Field	Value
Author	Netanel Yevarkan
Version	2.2 — Final
Document Type	Business Requirements Document (BRD)
Project Type	Self-Initiated Portfolio Artifact
Project Origin	Self-initiated, based on 11 years of field experience in passenger rail operations
Date	May 2026
Status	Final — Submitted as Portfolio Artifact

About This Document

This Business Requirements Document is a self-initiated portfolio artifact authored by Netanel Yevarkan. It is not commissioned, endorsed, or reviewed by any organization, and it does not represent the official position, plans, or data of any specific employer.

The author is a passenger train driver with 11 years of operational experience and a final-year B.A. student in Business Administration & Information Systems. The document draws on the author's domain knowledge of passenger rail operations to construct a realistic but illustrative requirements specification for a hypothetical mobile driver portal.

All organizational details, system names, integration touch-points, headcounts, and ROI figures presented in this document are **illustrative estimates by the author** for the purpose of demonstrating Business Analyst capability. They are not derived from any specific organization's confidential data and should not be treated as such.

The work product is intended to demonstrate the author's ability to: identify a domain problem, structure functional and non-functional requirements, design a logical data model, propose system integrations, build a quantitative business case, and frame a phased implementation roadmap.

1. Executive Summary

In a typical passenger rail operator at national scale, several hundred train drivers receive shift schedules, route changes, and operational updates through a fragmented mix of email, SMS, and informal messaging channels — often with no unified, real-time digital interface. This fragmentation results in avoidable miscommunications, manual coordination overhead for dispatch teams, and drivers operating without full situational awareness.

The Driver Portal proposed in this document is a mobile-first digital platform that consolidates shift management, real-time operational communication, and taxi coordination into a single application for drivers and dispatch teams.

This BRD defines the functional requirements, proposed system integrations, success metrics, and business case for the project. It was authored independently as both a portfolio artifact and a hypothetical proposal grounded in real domain experience.

2. Problem Statement — Current State

The current operational environment, as observed by the author across years of field experience, presents three interconnected pain points:

2.1 Shift Schedule Fragmentation

- Shift schedules are distributed via email, printed boards, and informal messaging groups
- No single source of truth — drivers must check multiple channels to confirm their schedule
- Last-minute changes are communicated informally, creating version-control problems
- Drivers in remote depots have delayed access to updates

2.2 Taxi Coordination Inefficiency

- Taxi pickups for early/late shifts are coordinated manually by dispatch via phone calls and SMS
- Drivers have no visibility into confirmed pickup time until shortly before departure
- No digital record of taxi requests — disputes and missed pickups are common
- Dispatch teams spend significant time on coordination that could be automated

2.3 Communication Fragmentation

- No structured channel for drivers to submit queries, flag operational issues, or receive updates
- Informal messaging groups are unstructured, noisy, and lack organizational governance
- Critical safety-adjacent communications (route changes, track restrictions) are mixed with social messages
- No read-receipt or acknowledgment mechanism for critical updates

3. Gap Analysis

Challenge	Current State	Impact	Proposed Solution
Schedule Access	Email + printed boards + messaging	Errors, confusion, driver anxiety	Unified schedule view with push notifications
Taxi Coordination	Manual phone/SMS dispatch	Missed pickups, manual overhead	In-app taxi request + status tracking
Operational Updates	Messaging groups (unstructured)	Critical info lost in noise	Structured notification center + read receipts
Driver Queries	Phone calls to dispatch	Dispatch bottleneck, delays	In-app Q&A + AI-assisted FAQ responses
HR / Leave Visibility	Separate HR system login	Low engagement, incomplete requests	Integrated leave balance via API

4. Functional Requirements

4.1 Shift Management Module

- Display current and upcoming shifts for the authenticated driver
- Push notification for schedule changes (minimum 2 hours in advance)
- Acknowledge/confirm shift receipt — with read-receipt logged to dispatch
- Filter view by depot, route, date range
- Export shift summary (PDF or iCal format)

4.2 Taxi Request Module

- Submit taxi pickup request with: shift ID, pickup address, requested time
- Real-time status tracking: Requested → Confirmed → En Route → Arrived
- Push notification at each status change
- Cancel request (up to 2 hours before pickup)
- History log of past requests

4.3 Communication & Notifications Center

- Structured inbox for operational updates from dispatch
- Priority levels: Informational / Operational / Urgent
- Read receipt required for Urgent messages
- Driver can flag a message as 'Requires Response'

- Admin panel for dispatch to compose and target messages (all drivers / by depot / individual)

4.4 HR Integration — Leave & Balance

- View remaining leave balance (annual, sick, emergency) — sourced from the HR system of record via API
- Submit leave request form — routed to line manager workflow
- View leave request status
- Display does not allow editing HR records directly — read + request only

4.5 Driver Knowledge Base (Phase 2)

- Searchable FAQ covering: regulations, procedures, emergency protocols
- AI-assisted natural language query — e.g., 'What is the procedure for brake failure at a major terminal station?'
- Content managed by Safety & Operations teams
- Version-controlled — all drivers see the same current version

5. System Integrations

System	Integration Type	Data Flow	Technical Notes
Scheduling System	REST API (JSON)	Read-only pull	Polling every 5 min; batch fallback from spreadsheet export if API unavailable
Dispatch Operations System	REST API (JSON)	Read + Write	API polling every 15 minutes for status updates; driver app writes taxi requests
HR System of Record	SSO + REST API	Read-only (leave balance)	Authentication via SSO; leave balance via REST API endpoint; no write access to HR records
Push Notifications	FCM / APNs	Server to device	Firebase Cloud Messaging (Android) + Apple Push Notification Service (iOS)
Authentication	SSO via organizational IDP	Login only	Leverage existing organizational identity provider; no separate credentials for drivers

6. User Stories

6.1 Driver User Stories

ID	As a driver, I want to...	So that...	Acceptance Criteria
US-01	View my confirmed shift schedule for the next 14 days	I can plan my personal life with certainty	Schedule displayed within 2 seconds of login; updated within 5 min of dispatch

ID	As a driver, I want to...	So that...	Acceptance Criteria
			change
US-02	Receive a push notification when my shift is changed	I am never surprised by a last-minute change	Notification delivered within 60 seconds of change; includes shift ID and new time
US-03	Confirm receipt of my shift schedule	Dispatch knows I have seen my assignment	One-tap acknowledgment; logged with timestamp
US-04	Request a taxi pickup for my upcoming shift	I don't need to call dispatch manually	Form submitted in under 30 seconds; confirmation received within 5 minutes
US-05	Track my taxi in real time	I know when to go downstairs	Status updates every 5 minutes; push notification on each state change
US-06	View my remaining annual leave balance	I can plan leave requests accurately	Balance shown on home screen; synced from HR system within 24 hours
US-07	Submit a leave request from the app	I don't need to visit the office or use a separate system	Request submitted and routed to manager within 2 minutes
US-08	Search the driver knowledge base	I can get answers to procedural questions without calling dispatch	Full-text search returns results in under 3 seconds

6.2 Dispatch / Admin User Stories

ID	As dispatch/admin, I want to...	Acceptance Criteria
US-09	Send an operational update to all drivers or a specific depot	Message delivered and read receipts logged within 5 minutes
US-10	See which drivers have acknowledged a critical update	Real-time dashboard showing read/unread per driver
US-11	Confirm or reject a taxi pickup request from the app	Response delivered to driver within 5 minutes of submission
US-12	View a log of all driver queries submitted this week	Filterable by depot, date, status — exportable to spreadsheet

7. Process Flows — As-Is vs. To-Be

7.1 Shift Change Communication

AS-IS (Current State)	TO-BE (Driver Portal)
1. Scheduler updates spreadsheet 2. Dispatcher contacts driver via messaging/phone 3. Driver manually checks multiple channels 4. No acknowledgment mechanism 5. Dispatch has no confirmation driver received update	1. Scheduler updates system — API push to Driver Portal 2. Driver receives push notification within 60 seconds 3. Driver taps to view updated schedule + confirm 4. Acknowledgment logged with timestamp 5. Dispatch dashboard shows confirmed/pending in real time

7.2 Taxi Coordination

AS-IS (Current State)	TO-BE (Driver Portal)
1. Driver calls dispatch to request taxi 2. Dispatch contacts taxi company manually 3. Driver waits for callback confirmation 4. No digital record — disputes unresolvable	1. Driver submits request via app in under 30 sec 2. Request auto-routed to dispatch dashboard 3. Dispatch confirms via one tap — push to driver 4. All requests logged with full audit trail

8. Non-Functional Requirements

8.1 Performance

- Application load time: under 3 seconds on standard 4G connection
- API response time: under 500ms for schedule and notification queries
- Support for several hundred concurrent users (designed to scale further in Phase 3)
- Push notification delivery: within 60 seconds of trigger event

8.2 Availability & Reliability

- System uptime: 99.5% minimum (excluding planned maintenance windows)
- Offline mode: schedule data cached locally for up to 24 hours
- Graceful degradation: core schedule view available even when integrations are down
- Maintenance windows: scheduled for 02:00–04:00 (lowest operational activity)

8.3 Data Privacy & Security

- All data transmitted over HTTPS (TLS 1.2 minimum)
- Role-based access control (RBAC): drivers see only their own data; dispatch sees assigned depot
- No HR data (leave balances, personal details) stored in app database — fetched live and not cached

- Driver activity logs (login, acknowledgments) retained for 90 days, then auto-deleted
- Aligned with applicable privacy regulations (e.g., GDPR-equivalent national privacy law) — formal legal review required prior to deployment
- Data Processing Agreement (DPA) required with any third-party vendor (taxi, notifications)
- No biometric data collected at any phase

8.4 Usability

- Mobile-first design — iOS and Android; no desktop requirement
- Hebrew-language interface (primary); Arabic secondary for non-Hebrew-speaking drivers
- Accessibility: WCAG 2.1 AA compliance for text contrast and touch targets
- Onboarding: new driver can reach first confirmed shift view within 5 minutes of first login

9. Business Case & ROI Estimates

Important: The following figures are **illustrative estimates by the author**, constructed for the purpose of demonstrating ROI modeling capability. They are not derived from any specific organization's payroll, dispatch, or operational data. Any real-world implementation would require formal measurement and validation against actual operational data before being used for budget decisions.

9.1 Dispatch Efficiency (Taxi Coordination)

Parameter	Illustrative Current State	Illustrative Post-Portal
Manual taxi coordination calls per day	~60 calls/day (avg. 4 min each)	~10 calls/day (exceptions only)
Dispatcher time saved per day	—	~200 minutes (~3.3 hours)
Annual dispatcher hours saved	—	~800 hours/year
Illustrative annual cost saving (dispatcher)	—	~₪84,000/year*

**Based on an illustrative dispatcher cost of ~₪105/hour. Any real-world figure requires validation against actual payroll data of the deploying organization.*

9.2 Driver Time Savings

Parameter	Current Estimate	Post-Portal Estimate
Time per driver checking schedule across channels (daily)	~8–12 minutes/day	~2–3 minutes/day
Aggregate driver time saved (several hundred drivers)	—	~5,000+ hours/year (illustrative)

9.3 Qualitative Benefits

- Reduced driver anxiety around shift uncertainty → lower unplanned absences
- Fewer missed pickups → improved on-time departure rates
- Formal communication trail for operational changes → reduced grievance escalations
- Management visibility into acknowledgment rates → earlier intervention for communication gaps

10. Entity Relationship Diagram (Logical)

The following entities and relationships define the core data model:

10.1 Core Entities

Entity	Key Attributes	Relationships
Driver	driver_id (PK), employee_id, name, depot, role, phone, language_pref	Has many Shifts; Has many TaxiRequests; Has many Messages (received); Has many AcknowledgmentLogs
Shift	shift_id (PK), driver_id (FK), route_id (FK), start_time, end_time, depot, status, last_updated	Belongs to Driver; Belongs to Route; Has many Notifications
TaxiRequest	request_id (PK), driver_id (FK), shift_id (FK), pickup_address, pickup_time, status, created_at, confirmed_by	Belongs to Driver; Linked to Shift; Has many StatusLogs
Message	message_id (PK), sender_id (FK), target_scope, priority_level, body, sent_at, requires_ack	Has many AcknowledgmentLogs; Created by Admin/Dispatch
AcknowledgmentLog	log_id (PK), driver_id (FK), message_id (FK), acknowledged_at, method	Belongs to Driver; Belongs to Message
LeaveRequest	request_id (PK), driver_id (FK), leave_type, start_date, end_date, status, submitted_at	Belongs to Driver; Routed to line manager workflow

11. Risk Register

Risk	Likelihood	Impact	Mitigation
API access to scheduling system not granted	Medium	High	Begin with manual spreadsheet export as fallback; escalate API access as Phase 1.5 requirement
Driver adoption resistance (habit change)	Medium	Medium	Pilot with volunteer drivers at one depot; gather feedback; incentivize early adoption
HR SSO integration complexity	Medium	Medium	Engage IT team early; design read-only endpoint as minimum viable integration; leave balance can be manually entered as fallback
Data privacy compliance	Low	High	Legal review before launch; no HR

Risk	Likelihood	Impact	Mitigation
gap			data cached; DPA with all vendors; quarterly compliance audit
Network reliability at remote depots	Medium	Low	Offline-first design; schedule data cached for 24 hours; push notifications queued on server until connection restored
Scope creep in Phase 1	High	Medium	Strict MVP scope defined in this BRD; Phase 2 features formally gated behind Phase 1 user acceptance

12. Phased Implementation Roadmap

Phase	Timeline	Scope	Success Gate
Phase 1 — MVP	Months 1–4	Shift schedule view, push notifications, taxi request, basic messaging center	90% daily active usage; 30% reduction in dispatch calls
Phase 2 — Enhancement	Months 5–8	HR leave integration, driver knowledge base, AI-assisted FAQ, read receipts dashboard	4.5+ avg. user satisfaction score; leave request volume shift to app
Phase 3 — Scale	Months 9–14	Expand to additional driver populations (e.g., freight, inspectors); fault reporting interface; Arabic language support	Several hundred additional active users; fault reporting integrated with technical center ticketing
Phase 4 — Intelligence	Months 15+	Predictive scheduling alerts, analytics dashboard for operations management, mobile fault photo upload	TBD based on Phase 3 outcomes

13. Cross-Domain Applicability

While the Driver Portal is designed for operational driver workflows, its architecture and core modules are directly applicable to adjacent domains within a rail organization:

13.1 Adjacent Use Cases

- Real-time fault reporting: drivers can report locomotive or rolling stock faults directly from the app — including photo capture, automatic location detection, and direct ticket creation in the technical maintenance system
- Maintenance schedule visibility: drivers can view scheduled maintenance windows for their assigned train set — reducing conflicts between operational scheduling and maintenance

- Direct communication channel with maintenance teams: structured messaging between field drivers and depot maintenance crews, replacing informal contact
- Fleet utilization data: driver check-in data combined with shift data provides operational management with real-time availability insights

13.2 Architectural Implications

These adjacent use cases share the same architectural backbone (authentication, push notifications, API integration with operational systems, audit logging). Designing the Driver Portal MVP with extensibility in mind reduces the marginal cost of adding adjacent modules in later phases.

14. Proposed Success Metrics

The following KPIs are proposed targets for Phase 1 (MVP) evaluation. Baselines should be established during a pre-launch measurement period.

Metric	Baseline	Target (Month 4)	Measurement Method
Daily Active Usage rate (DAU / total drivers)	0% (no app)	90%+	App analytics (login events)
Dispatch calls for taxi coordination	~60/day (estimated)	< 20/day	Dispatch call log
Shift acknowledgment rate	~0% (no mechanism)	95%+	In-app acknowledgment log
User satisfaction score (CSAT)	N/A	4.5 / 5.0	In-app monthly survey
Average schedule-check time per driver	~10 min/day	< 3 min/day	Self-reported + observational
Missed taxi pickups per month	~8–12 (estimated)	< 2	Dispatch incident log

Note: All targets are marked as 'proposed' — they reflect the author's operational judgment and would require agreement with operations management and IT leadership before being adopted as formal project KPIs in any real-world deployment.

15. Assumptions & Out of Scope

15.1 Assumptions

- The deploying organization's IT team would provide API documentation for scheduling and dispatch systems
- The HR system of record supports REST API for leave balance queries (or an equivalent integration point)
- Organizational SSO (Active Directory or equivalent) can be extended to mobile authentication
- Drivers have access to personal smartphones capable of running a modern mobile app
- Senior leadership sponsors Phase 3 expansion to adjacent driver populations

15.2 Out of Scope (Phase 1)

- Integration with locomotive control systems (SCADA, safety systems)
- Financial or payroll data beyond leave balance display
- Desktop or web-browser version of the portal
- Automated dispatch decision-making or scheduling AI
- Adjacent-population driver functionality (Phase 3)

Appendix — Document Version History

Version	Date	Changes	Author
1.0	December 2025	Initial draft — core functional requirements, gap analysis, user stories	Netanel Yevarkan
2.0	January 2026	Added ERD, process flows, risk register, phased roadmap, cross-domain applicability section	Netanel Yevarkan
2.1	March 2026	Language calibration — KPIs reframed as 'proposed metrics'; scalability as 'identified potential'	Netanel Yevarkan
2.2	May 2026	Added: API/integration technical details (§5), Data Privacy section (§8.3), ROI/Business Case chapter (§9)	Netanel Yevarkan

This document was authored as part of a career-transition portfolio. All operational data points are based on field observations and reasonable estimates by the author; they are illustrative and not validated against any specific organization's confidential data.